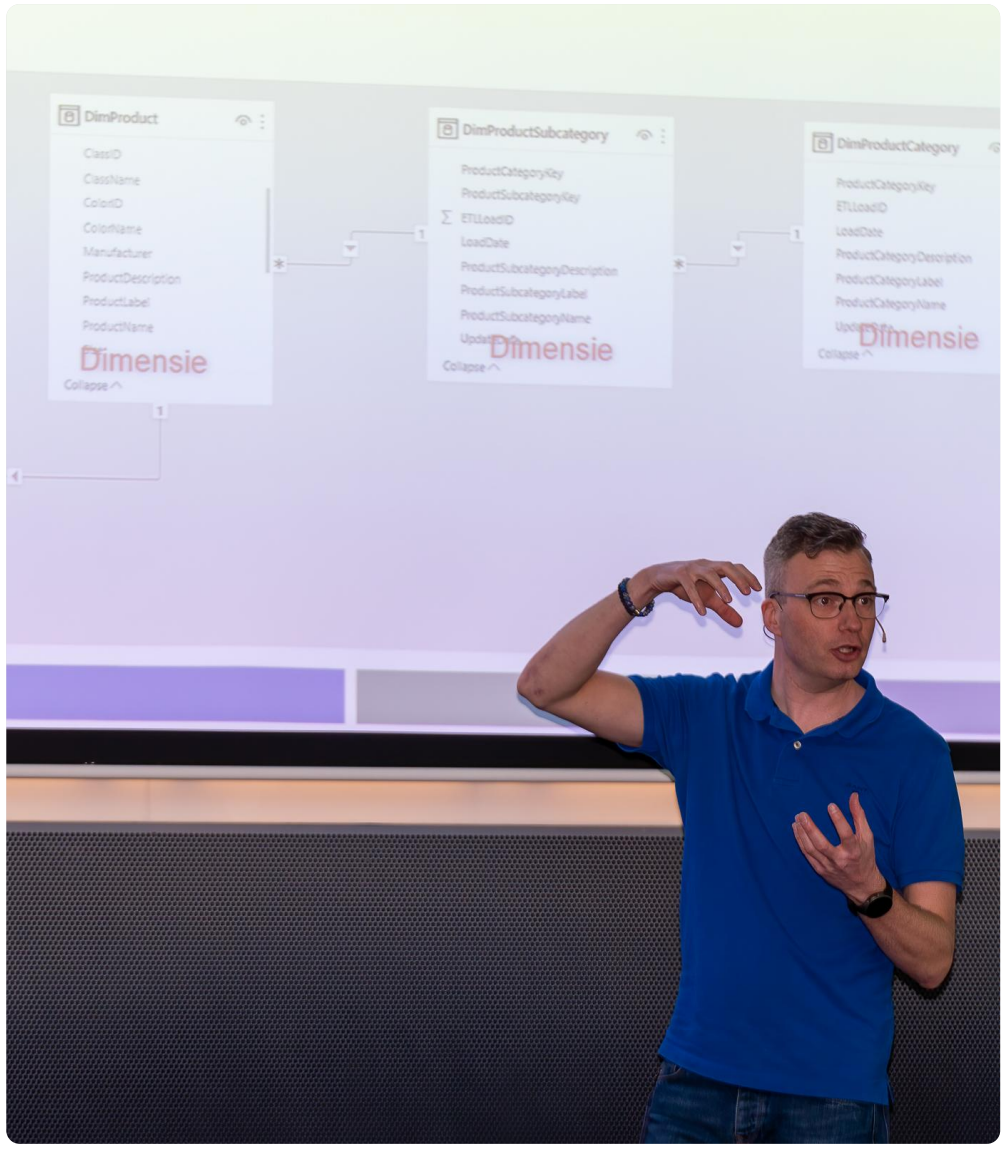
dutchBIgeek

Practical Data Modeling in Power BI: Beyond the Star Schema

11-03-2026



Peter van den Bos

Data Modeling & DAX Specialist
Freelance Trainer & Speaker
Founder @ DutchBiGeek

dutchbigeek.nl
linkedin.com/in/petervandenbos
github.com/PetervandenBos/
sessionize.com/peter-van-den-bos
youtube.com/@DutchBIgeek
peter@dutchbigeek.nl

🧠 **Geek by design**

⚡ *Because performance isn't optional*

Why is the Data Model Important?

- The data model is the foundation of Power BI.
- A well-designed data model improves performance and speeds up queries.
- 80% of performance issues are related to the data model.
- A better data model makes DAX queries easier and more efficient.
- Reduces data redundancy and minimizes errors.

Three Common Modeling Problems

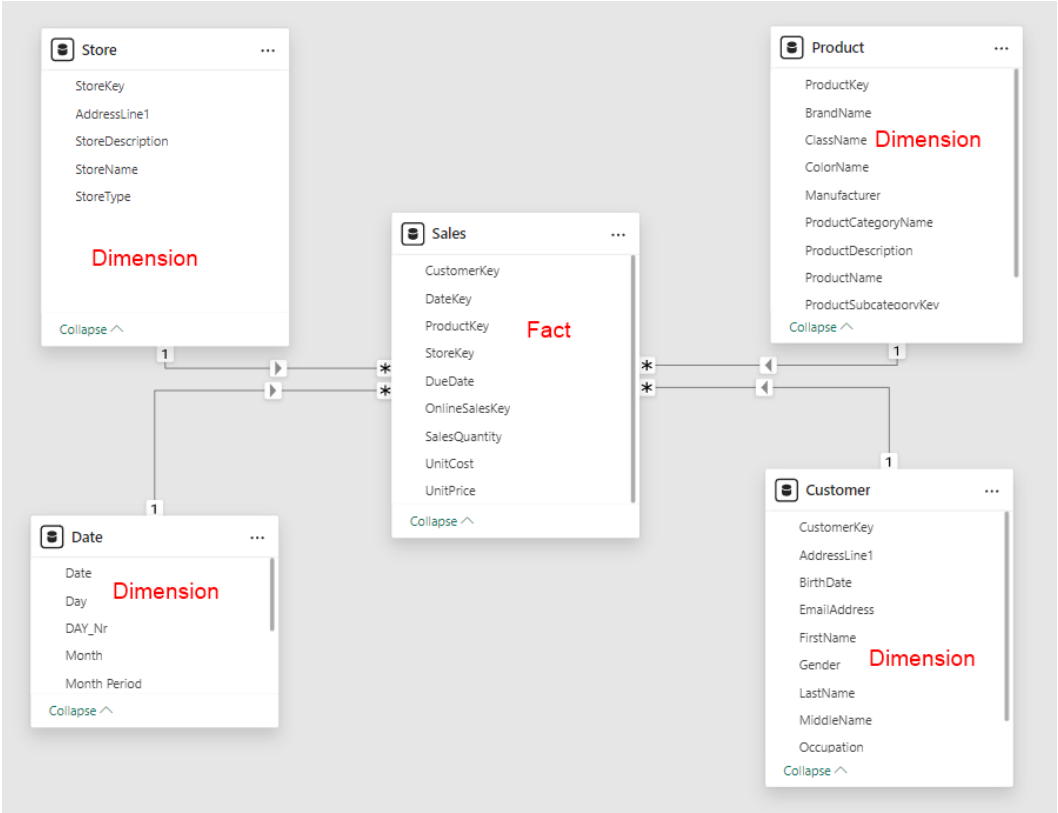
- Grain not defined
- Wrong fact table type
- Many-to-many hiding modeling problems

Star Schema

Data Modeling

- **Fact Tables:**
  - Contain facts, representing an event with dimensions.
  - A sale includes a product, a customer, and a date.
  - Metrics that can be aggregated to gain insights.

- **Dimension Tables:**
  - Descriptive attributes of entities such as a product, customer, employee, or patient.
  - Dimensions have attributes like color, category, manufacturer, or price.



dutchBIgeek

Snowflake-model

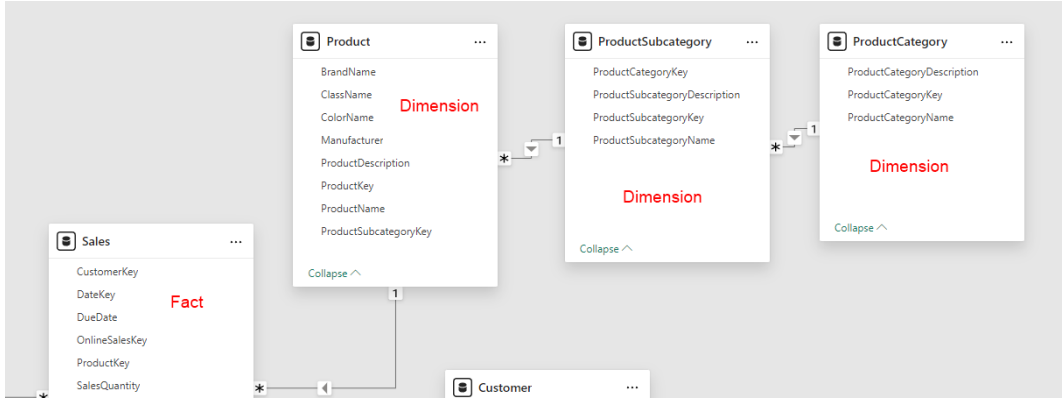
Data Modeling

The Snowflake Schema is a Variant of the Star Schema

The difference is that dimension tables that are related to each other are connected.

For example:

- Product
- Product Subcategory
- Product Category



dutchBIgeek

Granularity Determines Your Model

Before building the model, you must answer:

- What business question must be answered?
- What business event are we measuring?
- What does one single row represent?

Why This Comes First

- The grain determines the dimensions
- The grain determines valid measures
- The grain determines the fact table type
- If the grain is unclear, the model will be unstable.

What Goes Wrong When Granularity Is Wrong?

Data Modeling

When Grains Do Not Align

- Different levels of detail are compared directly
- Aggregations become misleading
- Totals appear correct — but are conceptually wrong

Example: Monthly Target vs Daily Sales

- Target defined at month level
- Sales recorded at day (or transaction) level
- Two different grains
- Repeating the monthly target per day inflates totals
- Comparing without adjustment leads to incorrect conclusions

How To Choose the Right Granularity

Start With the Business Question

- What question must be answered?
- What decision needs to be supported?

The question defines the grain.

The grain follows the business question — not the available data.

Granularity Drives Fact Type

Transaction Fact

- One row per business event
- Detailed, event-driven data

Snapshot Fact

- One row per entity per time period
- Captures status at a specific moment

Accumulating Snapshot Fact

- One row per process instance
- Tracks milestones within a lifecycle

Example 1 – Sales

- If we want to know total sales per day
- The grain is day
- Not every individual transaction

One row represents sales at the required reporting level.

Transaction Fact

Data Modeling

Definition

- One row per business event
- Grain

Atomic level of detail

- Clearly defined business event
- No mixed levels of detail

Structure

Foreign keys to dimensions

- Numeric, usually fully additive measures
- Large number of rows

A transaction fact stores the lowest meaningful level of detail.

Order Number	Line Number	Order Date	Delivery Date	CustomerKey	StoreKey	ProductKey	Quantity	Unit Price	Net Price	Unit Cost	Currency Code	Exchange Rate
370516	0	Saturday, 22 February 2020	Tuesday, 25 February 2020	1891277	999999	1730	1	€ 77,68	€ 71,4656	€ 35,72	USD	1
370909	0	Wednesday, 26 February 2020	Saturday, 29 February 2020	1463606	999999	1549	1	€ 389	€ 365,66	€ 128,88	USD	1
371002	0	Thursday, 27 February 2020	Monday, 2 March 2020	1683913	999999	1639	1	€ 9,99	€ 8,7912	€ 5,09	USD	1
371010	0	Thursday, 27 February 2020	Monday, 2 March 2020	1226217	999999	1631	1	€ 12,66	€ 12,66	€ 5,82	USD	1
370204	0	Wednesday, 19 February 2020	Thursday, 20 February 2020	1230737	999999	1656	1	€ 159,99	€ 142,3911	€ 73,57	USD	1
370211	0	Wednesday, 19 February 2020	Sunday, 23 February 2020	1402732	999999	468	1	€ 179	€ 166,47	€ 82,32	USD	1
370212	0	Wednesday, 19 February 2020	Friday, 21 February 2020	1601713	999999	145	1	€ 2.899,99	€ 2.551,9912	€ 960,82	USD	1
370309	0	Thursday, 20 February 2020	Friday, 21 February 2020	1790900	999999	680	1	€ 116	€ 116	€ 53,34	USD	1
370400	0	Friday, 21 February 2020	Sunday, 23 February 2020	1326869	999999	1802	1	€ 32	€ 29,76	€ 16,31	USD	1
370401	0	Friday, 21 February 2020	Saturday, 22 February 2020	1881362	999999	1705	1	€ 6,88	€ 6,88	€ 3,16	USD	1
370402	0	Friday, 21 February 2020	Wednesday, 26 February 2020	1520875	999999	1580	1	€ 219	€ 190,53	€ 72,56	USD	1
370406	0	Friday, 21 February 2020	Sunday, 23 February 2020	1525057	999999	2494	1	€ 2,94	€ 2,793	€ 1,5	USD	1
370506	0	Saturday, 22 February 2020	Tuesday, 25 February 2020	1285586	999999	1323	1	€ 35,99	€ 30,9514	€ 16,55	USD	1
369604	0	Thursday, 13 February 2020	Saturday, 15 February 2020	1413494	999999	1643	1	€ 57,88	€ 57,88	€ 26,62	USD	1
369607	0	Thursday, 13 February 2020	Saturday, 15 February 2020	1972546	999999	426	1	€ 499,9	€ 434,913	€ 254,86	USD	1
369609	0	Thursday, 13 February 2020	Saturday, 15 February 2020	1399286	999999	119	1	€ 119,99	€ 107,991	€ 61,17	USD	1
369703	0	Friday, 14 February 2020	Monday, 17 February 2020	1298824	999999	468	1	€ 179	€ 157,52	€ 82,32	USD	1
369706	0	Friday, 14 February 2020	Monday, 17 February 2020	1833911	999999	1825	1	€ 32	€ 32	€ 16,31	USD	1
370002	0	Monday, 17 February 2020	Wednesday, 19 February 2020	1919459	999999	1600	1	€ 57,88	€ 57,88	€ 26,62	USD	1
368406	0	Saturday, 1 February 2020	Monday, 3 February 2020	1582048	999999	505	1	€ 619	€ 619	€ 205,09	USD	1
368903	0	Thursday, 6 February 2020	Friday, 7 February 2020	1973637	999999	1629	1	€ 9,99	€ 9,2907	€ 5,09	USD	1
369106	0	Saturday, 8 February 2020	Wednesday, 12 February 2020	2081772	999999	1471	1	€ 229	€ 206,1	€ 105,31	USD	1
369108	0	Saturday, 8 February 2020	Saturday, 15 February 2020	1726873	999999	1597	1	€ 57,88	€ 51,5132	€ 26,62	USD	1
369301	0	Monday, 10 February 2020	Thursday, 13 February 2020	1476411	999999	52	1	€ 199,95	€ 199,95	€ 91,95	USD	1
366803	0	Thursday, 16 January 2020	Sunday, 19 January 2020	1394357	999999	855	1	€ 129	€ 129	€ 59,32	USD	1
366901	0	Friday, 17 January 2020	Tuesday, 21 January 2020	1630626	999999	363	1	€ 699	€ 601,14	€ 321,44	USD	1
366903	0	Friday, 17 January 2020	Sunday, 19 January 2020	1512517	999999	1625	1	€ 219	€ 194,91	€ 72,56	USD	1
367001	0	Saturday, 18 January 2020	Monday, 20 January 2020	1907830	999999	1694	1	€ 8,88	€ 7,9032	€ 4,08	USD	1
367007	0	Saturday, 18 January 2020	Tuesday, 21 January 2020	1386969	999999	2490	1	€ 14,99	€ 14,99	€ 7,64	USD	1
367300	0	Tuesday, 21 January 2020	Saturday, 25 January 2020	1477839	999999	1138	1	€ 391,9	€ 348,791	€ 180,22	USD	1

dutchBIgeek

Example 2 – Employees

- If we want to know how many employees we have on a specific day
- And in which department they work

The grain becomes:
Employees – Day – Department

One row represents employees on one specific day in one department.

Snapshot Fact

Data Modeling

Definition

- One row per entity per time period

Grain

- Status at a specific moment in time
- One entity per defined period (e.g., day, month)
- Time-based consistency

Structure

- Foreign keys to dimensions
- Measures represent a state, not an event
- Often semi-additive

A snapshot fact captures a state — not an event.

SnapshotDate	DateKey	MedewerkerSK	FTE	Headcount
Sunday, 1 January 2023	20230101	1	1	1
Sunday, 1 January 2023	20230101	5	1	1
Sunday, 1 January 2023	20230101	9	1	1
Sunday, 1 January 2023	20230101	11	1	1
Sunday, 1 January 2023	20230101	12	1	1
Sunday, 1 January 2023	20230101	14	1	1
Sunday, 1 January 2023	20230101	15	1	1
Wednesday, 1 February 2023	20230201	1	1	1
Wednesday, 1 February 2023	20230201	5	1	1
Wednesday, 1 February 2023	20230201	9	1	1
Wednesday, 1 February 2023	20230201	11	1	1
Wednesday, 1 February 2023	20230201	12	1	1
Wednesday, 1 February 2023	20230201	14	1	1
Wednesday, 1 February 2023	20230201	15	1	1
Wednesday, 1 March 2023	20230301	1	1	1
Wednesday, 1 March 2023	20230301	5	1	1
Wednesday, 1 March 2023	20230301	9	1	1
Wednesday, 1 March 2023	20230301	11	1	1
Wednesday, 1 March 2023	20230301	12	1	1
Wednesday, 1 March 2023	20230301	14	1	1
Wednesday, 1 March 2023	20230301	15	1	1
Saturday, 1 April 2023	20230401	1	1	1

Example 3 – Order Process

Data Modeling

If we want to analyze:

- When an order was received
- When it was shipped
- When it was delivered

The grain is one order (or shipment)

**We store milestones in one row —
not three separate rows.**

Accumulating Snapshot Fact

Data Modeling

Definition

- One row per process instance

Grain

- One business process
- Multiple milestones tracked in a single row
- Updated as the process progresses

Structure

- Foreign keys to dimensions
- Multiple date columns (milestones)
- Measures often represent durations or cycle times

An accumulating snapshot tracks progress — not repeated events.

OrderKey	OrderID	CustomerKey	ProductKey	OrderDateKey	ShipDateKey	DeliveryDateKey	OrderAmount	DaysToShip	DaysToDeliver	Status
1	ORD-001	4	1	20250128	20250129	20250130	210	1	2	Delivered
2	ORD-002	4	2	20250111	20250116	20250118	806	5	7	Delivered
3	ORD-003	2	1	20250106			954			Ordered
4	ORD-004	2	1	20250213	20250218	20250219	346	5	6	Delivered
5	ORD-005	3	2	20250121	20250124		873	3		Shipped
6	ORD-006	1	3	20250105	20250107		899	2		Shipped
7	ORD-007	5	4	20250110	20250111		381	1		Shipped
8	ORD-008	3	4	20250227			576			Ordered
9	ORD-009	3	2	20250209			881			Ordered
10	ORD-010	3	2	20250104	20250107	20250110	286	3	6	Delivered
11	ORD-011	3	5	20250217	20250222		653	5		Shipped
12	ORD-012	4	1	20250109	20250114	20250118	865	5	9	Delivered
13	ORD-013	4	1	20250114	20250118	20250120	437	4	6	Delivered
14	ORD-014	1	3	20250218			944			Ordered
15	ORD-015	3	3	20250101	20250104		712	3		Shipped
16	ORD-016	1	3	20250107	20250108	20250109	569	1	2	Delivered
17	ORD-017	1	2	20250226			538			Ordered
18	ORD-018	5	3	20250129	20250130		312	1		Shipped
19	ORD-019	5	5	20250127	20250129		569	2		Shipped
20	ORD-020	4	2	20250127	20250129	20250202	337	2	6	Delivered
21	ORD-021	1	2	20250118			712			Ordered
22	ORD-022	1	2	20250226	20250228	20250303	894	2	5	Delivered
23	ORD-023	4	5	20250108	20250112		964	4		Shipped
24	ORD-024	1	4	20250219	20250223	20250227	739	4	8	Delivered
25	ORD-025	3	4	20250106			256			Ordered
26	ORD-026	2	2	20250108			154			Ordered
27	ORD-027	4	5	20250121	20250123		662	2		Shipped

dutchBIgeek

When Granularity and Fact Types Aren’t Enough: Enter SCD Type 2

Data Modeling

Even With the Correct Grain…

- The fact table is correctly defined
- The grain is clear
- The fact type is appropriate

And still… results can be wrong.

Why?

- Dimensions change over time
- Historical context is lost
- Facts are evaluated against current dimension values

What Are Slowly Changing Dimensions (SCD)?

Data Modeling

Definition

- A dimension that changes over time.

Why It Matters

- Dimension attributes are not always static
- Business entities evolve
- Historical reporting requires context

Examples

- Employee changes department
- Product changes category
- Customer moves to a new region

Core Question

Do we overwrite history — or preserve it?

The Three Types of SCD

Data Modeling

Type 1 – Overwrite

- Old value is replaced
- No history is preserved
- Only the current state exists

Type 2 – Preserve History

- New row for each change
- Historical versions are stored
- Facts link to the correct version

Type 3 – Limited History

- Previous value stored in additional column
- Only limited history retained

Enter SCD Type 2

Data Modeling

What It Does

- Preserves history by creating a new row per change
- Each version gets its own Surrogate Key
- Facts link to the correct historical version

Required Columns (typical)

- SurrogateKey (technical key)
- BusinessKey (natural key, e.g. EmployeeID)
- ValidFrom / ValidTo (date range)
- IsCurrent (optional)

Why Type 2 Matters in Power BI

- Accurate historical slicing and grouping
- “Employee was in Department A at the time of the sale”
- Prevents today’s dimension values from rewriting history

What Are Many-to-Many Relationships (M2M)?

Data Modeling

Definition

A many-to-many relationship exists when:

- One value in Table A relates to multiple values in Table B
- And one value in Table B relates to multiple values in Table A

There is no single unique match.

In Dimensional Modeling

- Facts are typically many-to-one to dimensions
- M2M breaks this standard star schema pattern

Why It Matters in Power BI

- Ambiguous filter paths
- Double counting risks
- Complex DAX required
- Performance impact

Why M2M Breaks Your Model

Filter Propagation Becomes Harder to Control

- Filters can propagate across multiple related rows
- The relationship is no longer strictly many-to-one
- The result depends heavily on the evaluation context

Double Counting Risk

- A single fact can relate to multiple dimension rows
- Aggregations can be inflated
- Totals may look correct but be logically wrong

Grain Mismatch Is Often the Root Cause

- Data exists at different levels of detail
- Budget vs Sales is a common example
- The relationship compensates for a modeling issue

What Causes M2M?

Granularity Mismatch

- Data exists at different levels of detail
- Monthly budget vs daily sales
- Multiple facts sharing dimensions at different grains

M2M is often a symptom — not the root problem.

- Shared Ownership or Allocation
- One fact relates to multiple dimension members

Example:

- One sale linked to multiple sales representatives
- One project linked to multiple departments

Many-to-many relationships often indicate a modeling decision that needs reconsideration.

How to Fix M2M (1/2)

Data Modeling

Fix the Grain First

- Separate facts by granularity
- Align the reporting level before relating tables

Use a Bridge Table (when it’s truly M2M)

- Create an intermediate mapping table
- Fact → Bridge → Dimension
- Use single-direction filters

Use when:

- One transaction relates to multiple dimension members
- Many-to-one is impossible by design

How to Fix M2M (2/2)

Model Allocations Explicitly

- Convert M2M into a measurable allocation
- Store weights/percentages in a fact table
- Avoid repeating values across many rows

Example:

Budget allocated to departments by percentage

Data Modeling Best Practices

- Define the grain first – every model starts with granularity
- Choose the correct fact table type (transaction, snapshot, accumulating)
- Build a clear star schema
- Preserve history with SCD Type 2 when needed
- Avoid many-to-many relationships when possible
- Fix the grain before fixing relationships
- Use a bridge table only when M2M is unavoidable

dutchBIgeek

If you have questions or insights, please contact me !

Peter van den Bos

Business Intelligence Consultant

peter@dutchbigeek.nl

+31 6 13760795

https://www.linkedin.com/in/petervdbos/

dutchbigeek.nl